

TURNQUEST REPORTS

Overview

This document explains the architecture, database structure, XML templates, RTF templates, and report execution flow used in the TURNQUEST reporting system.

Table of Contents

1. [Introduction](#)
 2. [Core Report Tables](#)
 3. [Useful SQL Queries](#)
 4. [Oracle BI Publisher XML Structure](#)
 5. [XML Syntax Breakdown](#)
 6. [RTF Template Structure](#)
 7. [RTF Syntax Reference](#)
 8. [Report Generation Flow](#)
 9. [RunReport Java Method](#)
 10. [Dynamic Parameter Handling](#)
 11. [Helper Method](#)
-

Introduction

What is a System Report?

A **System Report** refers to unaudited internal monthly reports showing revenues and expenses for the business.

What is a Transaction Report?

A **Transaction Report** is a detailed record of a transaction that has occurred, for example:

- Debit Notes
 - Invoices
 - Policy Transactions
 - Financial Entries
-

Core Report Tables

The TURNQUEST reporting engine relies on several database tables.

Table Name	Description
TQC_SYSTEM_REPORTS	Stores XML report configuration information. Primary key: RPT_CODE
TQC_SYS_RPT_TEMPLATES	Stores report RTF template files. Linked using RPT_TMPL_RPT_CODE
TQC_SYS_RPT_PARAMETERS	Stores report parameters. Linked using RPTP_RPT_CODE
TQC_SYS_RPT_SUB_MODULES	Stores report submodules. Primary key: RSM_CODE

Relationships

- TQC_SYSTEM_REPORTS.RPT_CODE
- Links to TQC_SYS_RPT_TEMPLATES.RPT_TMPL_RPT_CODE
- Links to TQC_SYS_RPT_PARAMETERS.RPTP_RPT_CODE
- TQC_SYS_RPT_SUB_MODULES.RSM_CODE
- Linked through TQC_SYSTEM_REPORTS.RPT_RSM_CODE

Useful SQL Queries

```
SELECT * FROM TQC_SYSTEM_REPORTS;
SELECT * FROM TQC_SYS_RPT_TEMPLATES;
SELECT * FROM TQC_SYS_RPT_PARAMETERS;
SELECT * FROM TQC_SYS_RPT_SUB_MODULES;
```

Oracle BI Publisher XML Structure

An Oracle BI Publisher Data Template follows a strict XML syntax required by Oracle E-Business Suite / BI Publisher.

Standard XML Template Structure

```
<?xml version="1.0" encoding="UTF-8" ?>
<dataTemplate name="TemplateName"
  dataSourceRef="DatabaseRef"
  defaultPackage="PLSQL_PACKAGE_NAME">

  <properties>
    <property name="include_parameters" value="true/false"/>
    <property name="xml_tag_case" value="upper/lower/as_is"/>
  </properties>
</dataTemplate>
```

```

</properties>

<parameters>
  <parameter name="P_PARAM_NAME_1" dataType="character/number/date"/>
  <parameter name="P_PARAM_NAME_2"
    dataType="integer"
    defaultValue="10"/>
</parameters>

<lexicals>
  <lexical type="oracle.apps.bom.structure.Query"
    name="P_LEX_WHERE"
    comment="Dynamic where clause"/>
</lexicals>

<dataQuery>

  <sqlStatement name="Q_PARENT">
    <![CDATA[
      SELECT parent_id,
             parent_name,
             parent_date
      FROM parent_table
      WHERE parent_id = :P_PARAM_NAME_1
    ]]>
  </sqlStatement>

  <sqlStatement name="Q_CHILD">
    <![CDATA[
      SELECT child_id,
             fk_parent_id,
             child_name,
             child_amount
      FROM child_table
      WHERE fk_parent_id = :parent_id
    ]]>
  </sqlStatement>

</dataQuery>

<dataStructure>

  <group name="G_PARENT_ROOT" source="Q_PARENT">

    <element name="XML_TAG_ID"
      dataType="number"
      value="PARENT_ID"/>

    <element name="XML_TAG_NAME"
      dataType="varchar2"
      value="PARENT_NAME"/>
  </group>
</dataStructure>

```

```

<element name="XML_TAG_DATE"
        dataType="date"
        value="PARENT_DATE"/>

<group name="G_CHILD_ROWS" source="Q_CHILD">

    <element name="CHILD_ID"
            dataType="number"
            value="CHILD_ID"/>

    <element name="CHILD_NAME"
            dataType="varchar2"
            value="CHILD_NAME"/>

    <element name="TOTAL_AMOUNT"
            function="sum"
            dataType="number"
            value="CHILD_AMOUNT"/>

</group>

</group>

</dataStructure>

</dataTemplate>

```

XML Syntax Breakdown

<dataTemplate>

The mandatory root tag.

Attributes

Attribute	Description
name	Name of the template
dataSourceRef	Database/JNDI datasource
defaultPackage	Optional PL/SQL package

<parameters>

Defines input parameters.

Example

```
<parameter name="P_POLICY_NO" dataType="character"/>
```

Usage in SQL

```
WHERE policy_no = :P_POLICY_NO
```

<![CDATA[]]>

Used to wrap SQL statements so XML parsers ignore:

- <
- >
- &

<dataQuery>

Contains SQL statements.

Important Rule

Columns returned in SQL must match the values referenced later in **<element>** tags.

<dataStructure>

Defines the hierarchy of the generated XML.

<group>

Represents a repeating record loop.

Example

```
<group name="G_POLICIES" source="Q_POLICIES">
```

<element>

Maps XML nodes to SQL query columns.

Example

```
<element name="CLIENT_NAME"
  dataType="varchar2"
  value="CLIENT_NAME"/>
```

RTF Template Structure

Oracle BI Publisher RTF Templates define the visual layout for:

- PDFs
- Word Documents
- HTML Reports
- Printable Documents

RTF templates are usually designed in:

- Microsoft Word
- BI Publisher Word Add-in

The engine internally converts templates into:

- XSL-FO

RTF Syntax Reference

1. Simple Data Mapping

Syntax

```
<?XML_TAG_NAME?>
```

Example

```
<?CLIENT_NAME?>
<?POLICY_NUMBER?>
```

2. Repeating Loops

Used for dynamic rows like:

- Vehicles
- Transactions
- Premium Items
- Invoice Lines

Syntax

```
<?for-each:G_GROUP_NAME?>  
  ... content ...  
<?end for-each?>
```

Example

```
<?for-each:G_VEHICLES?>  
  <?REG_NO?>  
  <?SUM_INSURED?>  
<?end for-each?>
```

Notes

- Tags inside the loop must belong to the group.
 - In Word tables, place `for-each` in the first cell.
 - Place `end for-each` in the last cell.
-

3. Conditional Logic

Basic If Statement

```
<?if:CONDITION?>  
  ... content ...  
<?end if?>
```

Example

```
<?if:MOTOR='Y'?>  
Motor Policy  
<?end if?>
```

Numeric Example

```
<?if:PREMIUM > 0?>  
Premium Available  
<?end if?>
```

4. If / Else Logic

Syntax

```
<?choose:?>  
  <?when:CONDITION_1?>  
    ... content ...  
  <?end when?>  
  
  <?when:CONDITION_2?>  
    ... content ...  
  <?end when?>  
  
  <?otherwise:?>  
    ... default content ...  
  <?end otherwise?>  
<?end choose?>
```

5. Calculations and Summary Functions

Sum

```
<?sum(PREMIUM_AMOUNT)?>
```

Count

```
<?count(POLICY_NO)?>
```

Math Operations

```
<?FIELD_A + FIELD_B?>  
<?AMOUNT * 0.16?>
```

6. Formatting Numbers

Syntax

```
<?format-number:ELEMENT_NAME; '999,999,999.00'?>
```

Example

```
<?format-number:TOTAL_PREMIUM; '999,999,999.00'?>
```

7. Formatting Dates

Syntax

```
<?format-date:ELEMENT_NAME; 'DD-MON-YYYY'?>
```

Example

```
<?format-date:POLICY_DATE; 'DD-MON-YYYY'?>
```

Report Generation Flow

Oracle BI Publisher processes reports in 3 major stages.

Step 1 — XML Generation

The database executes SQL queries and generates hierarchical XML.

Step 2 — Layout Parsing

The RTF template is converted into XSL-FO formatting instructions.

Step 3 — Data Merge

The engine:

- Fills placeholders
- Executes loops

- Applies conditions
- Generates final output

Final Output Formats

- PDF
- DOC
- HTML
- Excel

RunReport Java Method

The `RunReport()` method is responsible for:

- Loading report metadata
- Loading templates
- Loading parameters
- Executing BI Publisher engines
- Returning downloadable output

Main Responsibilities

1. Fetch Report Details

```
String jobquery = "begin ? := tqc_web_cursor.getrptdetail(?); end;";
```

Fetches:

- Template File
- Style File
- Report Name
- XML Data File
- Printer Application

2. Load Dynamic Parameters

```
SELECT rptp_name  
FROM tqc_sys_rpt_parameters  
WHERE rptp_rpt_code = ?
```

Each parameter is:

- Retrieved dynamically
- Pulled from session/view state
- Stored in session

3. Execute Data Engine

```
xmlPublisher.dataEngine(reportName,  
                        dataFile,  
                        oracleConnection,  
                        this.session);
```

This generates XML data.

4. Execute Processor Engine

```
bytes = xmlPublisher.processorEngine(reportFormat,  
                                    templateFile,  
                                    styleFile,  
                                    reportName);
```

This merges:

- XML Data
- RTF Template
- Output Format

Into final downloadable bytes.

5. Stream Report to Browser

```
response.setHeader(  
    "Content-disposition",  
    "attachment; filename=" + output  
);
```

The generated report is downloaded automatically.

Dynamic Parameter Handling

Purpose

This section dynamically binds report parameters from:

- Database metadata
- Frontend/session values

Into BI Publisher.

Flow

```
Database Parameters
  ↓
Frontend/View State
  ↓
HTTP Session
  ↓
BI Publisher Engine
```

Dynamic Parameter Logic

```
while (paramRs.next()) {
    String paramName = paramRs.getString("rptp_name");

    Object paramValue = this.getViewParameterValue(paramName);

    if (paramValue != null) {
        this.session.setAttribute(paramName, paramValue);
    }
}
```

Explanation

Step	Purpose
Read parameter names	Fetch parameter metadata
Get UI value	Retrieve frontend value
Store in session	Make available to BI Publisher

Helper Method

`getViewParameterValue()`

This helper retrieves parameter values from:

- Session
- Request Parameters
- UI Components

Sample Stub

```
private Object getViewParameterValue(String paramName) {  
    return this.session.getAttribute(paramName);  
}
```

Real Usage Example

```
FacesContext.getCurrentInstance()  
    .getExternalContext()  
    .getRequestParameterMap()  
    .get(paramName);
```

Key Concepts Summary

Concept	Purpose
XML Template	Defines report data structure
RTF Template	Defines report visual layout
Data Engine	Generates XML data
Processor Engine	Generates final document
Parameters	Dynamic report inputs
Groups	Repeating data sections
Elements	XML field mappings
Session Attributes	Parameter transport mechanism

Best Practices

XML Templates

- Always wrap SQL with `<CDATA>`
- Ensure SQL columns match XML elements
- Use meaningful group names
- Keep XML hierarchy clean

RTF Templates

- Keep loops properly closed
- Avoid nested loops unless necessary

- Use format masks for numbers and dates
- Test conditional blocks thoroughly

Java Reporting Engine

- Always close DB resources
 - Use `finally` blocks
 - Validate parameters before execution
 - Handle null report formats safely
-

Conclusion

The TURNQUEST reporting framework integrates:

- Oracle BI Publisher
- XML Data Templates
- RTF Layout Templates
- Dynamic Java Report Execution
- Session-Based Parameter Passing

Together, these components provide a flexible enterprise reporting engine capable of generating:

- Financial Reports
- Policy Documents
- Transaction Reports
- Statements
- Operational Reports

in multiple output formats.